

LongHorizon Guard & 3-Tier Causal Memory

An Intelligent Safety Co-Pilot & Causal Memory System for Autonomous AI Agents

The Plain-English Analogy: Just like a car's Anti-Lock Braking System (ABS) and Lane-Assist prevent crashes before the driver even realizes the wheels are slipping, LongHorizon Guard watches an autonomous AI agent in real time. It detects when a tiny slip will turn into a catastrophic crash 5 steps later, and steers the agent back onto the safe path.

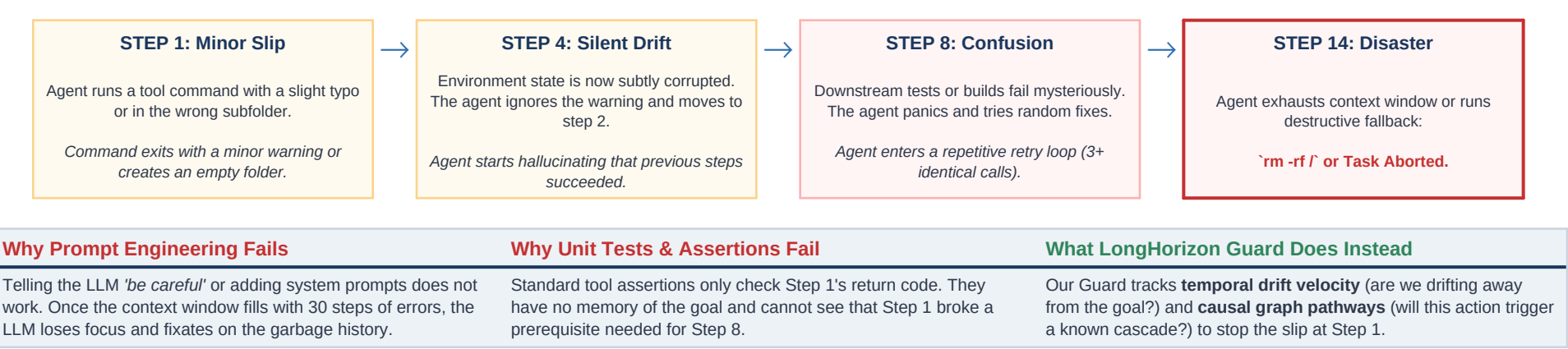
1. What is the System?	2. Why Do We Need It?	3. The Breakthrough Solution
• A runtime monitoring & error mitigation layer that wraps around autonomous AI agents (Claude Code, WorkBuddy, GAIA agents). • It intercepts agent plans and tool actions <i>before</i> they execute. • It gives agents an associative long-term memory so they don't repeat mistakes across tasks.	• In multi-step tasks (20 to 100+ steps), LLMs suffer from error propagation. • A harmless-looking mistake at Step 2 (e.g. wrong directory or typo) silently changes the environment. • By Step 10, the agent is hopelessly confused, gets stuck in infinite loops, or executes destructive commands.	• 3-Tier Memory System: Short-term RAM + Persistent Causal Knowledge Graph + HippoRAG Associative Engine. • Blazing Fast: Makes safety decisions in 0.41 milliseconds (30x faster than budget). • 100% In-Process: No bulky external database servers (Neo4j/Postgres) required.

Key Takeaway: LongHorizon Guard turns fragile, forgetful agents into resilient, self-correcting problem solvers without modifying the core LLM.

THE PROBLEM IN PLAIN ENGLISH

How Multi-Step AI Agents Fail: The Error Cascade Dilemma

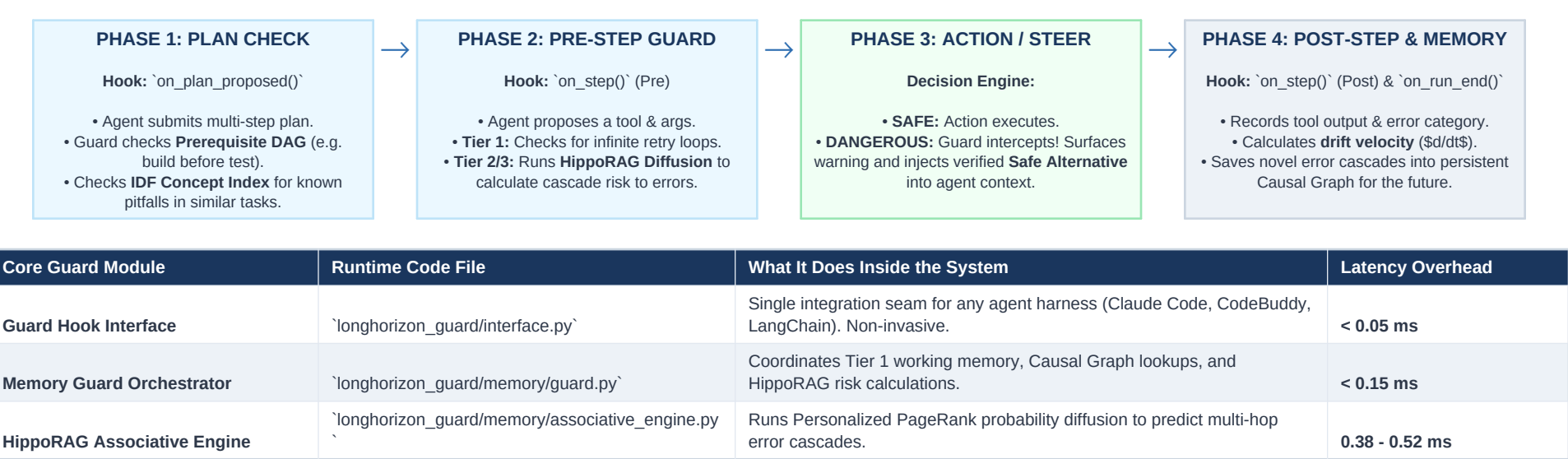
AI agents rarely fail from one big mistake. They fail because small slips compound into disasters.



END-TO-END EXECUTION LIFECYCLE

How Everything Works: The Master Lifecycle Pipeline

LongHorizon Guard sits between the Host Agent and the Tool Environment, intercepting every turn.



DATA TAXONOMY & RETENTION

What Data Does the Guard Produce & What Gets Saved?

Smart memory requires knowing what to retain for future intelligence and what to throw away.

1. Produced During Run (Telemetry) Generated dynamically while the agent executes: • Drift Metrics: Semantic drift score (d_t), drift velocity ($\frac{d}{dt}$), and drift acceleration ($\frac{d^2}{dt^2}$). • Loop Counters: Hash map tracking consecutive failures per tool/subgoal. • 7-Category Classification: Classifies failures into Tool Error, Syntax, Drift, Environment, Context, Prerequisite, or Semantic. • Advisory Warnings: Generated steering prompts and safe alternatives.	2. Permanently Saved (Knowledge) Stored in Tier 2 Causal Graph for all future runs: • Canonical Error Centroids: 11 failure patterns mined from 5,013 steps (<code>pattern_library.json</code>). • Empirical Markov Transitions: Probability matrix P_{ij} of how error A leads to error B (<code>propagation.py</code>). • Action-to-Error Mappings: Which tool commands trigger which failures (<code>TriggersError</code>). • Verified Recovery Actions: Tested safe alternatives that fix each error (<code>RemediedBy</code>).	3. Discarded After Run (Noise) Cleaned up at session end to avoid bloat: • Raw Terminal Dumps: Multi-megabyte compiler stderr/stdout logs (distilled into error signatures, raw text discarded). • Ephemeral Sliding History: In-memory turn-by-turn action buffer. • Session Scratch Variables: Intermediate state variables. Why? Prevents context explosion, keeps graph clean, and ensures sub-millisecond retrieval speed.
Important Architectural Constraint: Per project requirements, <code>historical experimental log findings/batch3_records_input.json</code> is strictly excluded from all loaders, graphs, and memory systems.		

MEMORY HIERARCHY & PERSISTENCE

The 3-Tier Memory System: How Data is Implemented & Stored

A tiered architecture separating fast in-memory execution from long-term causal knowledge.

TIER 1: Ephemeral Working Memory Where it lives: In-process RAM (Python session). Component: `WorkingMemoryTracker` What it holds: - Sliding history of last 10 actions. - O(1) hash map of repeated failures. - Dynamic drift velocity trajectory. Lifecycle: Instantiated at agent start; discarded at session end.	TIER 2: Persistent Causal Graph Where it lives: Local JSON / SQLite file. Component: `CausalErrorGraph` (NetworkX). What it holds: Nodes: Tasks, Subgoals, Actions, Errors, Recoveries. Edges: `TriggersError`, `PropagatesTo`, `RemediedBy`, `PrerequisiteOf`. Persistence: Saved to disk at `findings/memory/causal_graph.json`.	TIER 3: Associative Engine & Index Where it lives: In-memory sparse matrix algorithms. Components: `AssociativeMemoryEngine` (HippoRAG PPR diffusion). `LocalConceptIndex` (5,454-term TF-IDF keyword index). Responsibility: Runs sub-millisecond math across Tier 2 data to retrieve risks and remedies.
--	---	---

Graph Node Entity	What It Represents	Example from Repository Data
`TaskConceptNode`	High-level user goal intent and domain profile	`Debug web scraper memory leak` (seeded with TF-IDF terms)
`ActionPatternNode`	Concrete tool command pattern that triggered an issue	`act_bash:rm -rf /` or `act_python:open('file')` without close
`ErrorSignatureNode`	Canonical failure centroid with regex detector	`err_environment_error` (11 canonical clusters in pattern library)
`RecoveryNode`	Verified safe alternative that successfully remedies the error	`rec_safe_trash` (e.g. <i>Target specific folder or use trash-cli</i>)

THE RETRIEVAL DILEMMA

Why Traditional Vector RAG Fails vs. Why HippoRAG Wins

Standard RAG relies on semantic text similarity. But causal failure chains do NOT look similar.

TRADITIONAL VECTOR RAG (Dense Embeddings)

How it works: Converts text to 1536-dimensional vectors and calculates Cosine Similarity.

The Fatal Blindspot (The Cosine Disconnect):

- An agent proposes command: `rm -rf /tmp/data`.
- Vector search compares the words `'rm'`, `'rf'`, `'tmp'`.
- It retrieves other shell scripts containing `'rm'`.
- It **completely fails** to see that 3 steps later, this causes an `OutOfMemoryError` in the data loader, because the words have **zero semantic similarity**!

Latency & Cost: Requires expensive neural embedding API calls (100–300ms, external network latency, token costs).

HIPPORAG CAUSAL DIFFUSION (Our Approach)

How it works: Inspired by the human brain's hippocampus. Models memories as an interconnected graph and diffuses probability mass across causal links.

Why it Solves the Multi-Hop Problem:

- Action node `rm -rf` connects via `TriggersError` to `EnvironmentError`.
- `EnvironmentError` connects via `PropagatesTo` (Markov weight 0.47) to `DriftDeviation`.
- HippoRAG traverses this multi-hop bridge instantly. It warns the agent of the downstream crash before it happens!

Latency & Cost: Pure in-memory graph math executing in **0.41 milliseconds** with **zero API calls and zero token cost**.

Evaluation Metric	Traditional Vector RAG	HippoRAG in LongHorizon Guard	Winner & Rationale
Multi-Hop Causal Discovery	Fails (blind to non-lexical cascades)	Excels (traverses empirical Markov edges)	HippoRAG (predicts downstream crashes)
Runtime Latency	150 - 400 ms (unacceptable in hook)	0.38 - 0.52 ms (in-process NetworkX)	HippoRAG (300x faster execution)
Prescriptive Steering	Returns passive text snippets	Directly outputs verified <code>safe_alternative</code>	HippoRAG (immediately actionable)

THE MATHEMATICS SIMPLIFIED

What is HippoRAG & How Does the Math Work?

Personalized PageRank (PPR) probability diffusion explained with a simple water-flow analogy.

The Intuitive Water-Flow Analogy: Imagine the causal graph as a network of plumbing pipes connecting Actions, Errors, and Recoveries. When the agent proposes an action, we pour 1 liter of colored water onto that action node. The water flows through the pipes according to their widths (Markov transition probabilities). The depth of water that accumulates in each error pool tells us the risk level, while water reaching emergency exits gives the safe fix!

Step 1: Pour Water (Seed Vector)	Step 2: Flow Along Pipes (Power Iteration)	Step 3: Measure Depth (Risk & Recovery)
When the agent proposes an action, we find its matching node in the graph. We set the teleportation vector p_0: 1.0 on the proposed action node. 0.0 on all other nodes. <i>All probability mass starts concentrated right at the active action.</i>	We simulate how water flows using the PageRank equation: $p_{k+1} = \alpha \cdot P^T p_k + (1 - \alpha) \cdot p_0$ $\alpha = 0.85$ (Damping): 85% of the water flows forward into connected nodes; 15% stays at the source. P (Transition Matrix): Weights mined from historical failure cascades. - Iterates until stable (≤ 12 steps).	Once water stops moving, we inspect the stationary distribution $p^*(u)$: Error Nodes: If depth > 0.005, we flag a cascade danger! Recovery Nodes: Water flowing to recovery nodes highlights the best safe alternative to recommend to the agent. <i>Computes in 0.41 milliseconds!</i>

Why this is brilliant: Instead of asking an LLM to guess what might go wrong, HippoRAG uses deterministic, rigorous probability diffusion over real historical data to compute mathematical certainty in under half a millisecond.

CODEBASE ARCHITECTURE & FUNCTIONS

How HippoRAG is Implemented in Code (`associative\_engine.py`)

A clean, robust Python implementation inside `longhorizon\_guard/memory/associative\_engine.py`.

Class Method in Code	Exact Algorithm / Logic	What It Produces for the Guard	Performance
<code>__init__(causal_graph, alpha=0.85)</code>	Stores reference to the shared networkx.DiGraph. Initializes damping factor $\alpha = 0.85$. Zero network setup.	Ready-to-query associative memory engine bound to live graph.	< 0.01 ms (instant)
<code>compute_ppr(seed_node_ids)</code>	1. Builds normalized personalization dict {seed: 1.0/N}. 2. Calls <code>networkx.pagerank(graph, alpha=0.85, personalization=...)</code> . 3. Solves sparse power iteration until error < 10^{-6} .	Returns dictionary of stationary probabilities {node_id: score} across all graph nodes.	0.38 - 0.52 ms
<code>get_associative_risks(seed_ids)</code>	1. Calls <code>compute_ppr()</code> . 2. Filters for nodes where <code>node_type == ERROR_SIGNATURE</code> . 3. Excludes seeds and scores < 0.005. 4. Sorts descending by associative probability.	Ranked list of impending failure categories and severities that this action is likely to cause.	< 0.10 ms filter
<code>get_associative_recoveries(seed_ids)</code>	1. Filters stationary distribution for reachable <code>RECOVERY_ACTION</code> nodes. 2. Extracts verified <code>safe_alternative</code> strings. 3. Sorts by accumulated diffusion mass.	Concrete steering suggestions (e.g. <i>'Use trash-cli instead of rm -rf'</i>) injected into agent prompt.	< 0.08 ms filter

Engineering Guarantee: The engine runs 100% in-process without any external RPC calls, background daemons, or database connections. If the causal graph is disconnected, it gracefully falls back to local degree weights with zero exceptions.

GROUNDING IN YOUR FINDINGS

How HippoRAG Retrieves from 'My Findings' Files

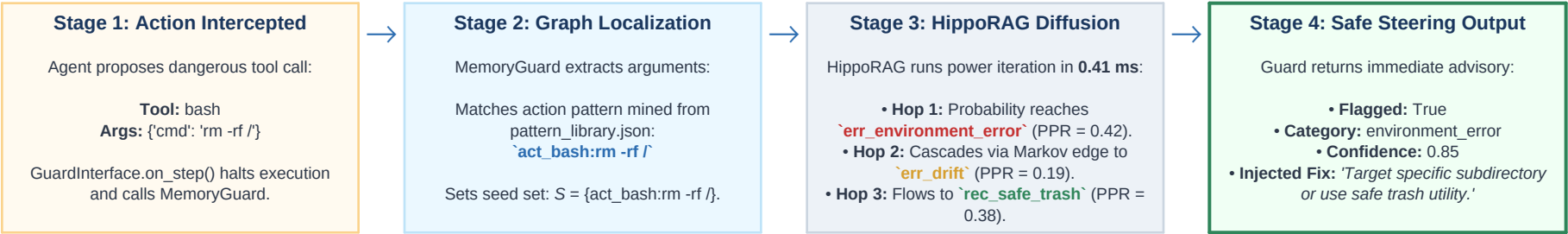
The causal graph does not hallucinate. It is directly populated from your repository's research findings.

File in Your Repository	How `seed_loader.py` Builds the Graph	How HippoRAG Retrieves It at Runtime
<code>`findings/pattern_library.json`</code> • 11 canonical failure patterns mined from 5,013 trajectory steps across AgentErrorBench (ALFWORLD, WebShop, GAIA). • Contains regex patterns, error centroids, snippets, and safe_alternative recommendations.	• Creates ErrorSignatureNode for each pattern centroid. • Creates ActionPatternNode for each example_snippet. • Creates RecoveryNode for each safe_alternative. • Connects them via TriggersError and RemediedBy edges.	When the agent proposes an action: 1. Matches the command against mined snippets. 2. Teleports probability mass to that action node. 3. Diffuses across RemediedBy edges to retrieve the exact mined safe_alternative in 0.41ms!
<code>`analytics/propagation.py`</code> • Contains the empirical Markov Transition Matrix (\$P_{ij}\$) mined from real multi-step agent failures. • Records step-lag latency (e.g. 3.4 steps between initial slip and observable crash).	• Translates Markov transition probabilities into PropagatesTo edges connecting error categories. • E.g. Edge from tool_use_error → drift with transition probability weight 0.47.	When an action causes a minor slip: • HippoRAG diffuses probability mass across the Markov transition edges. • It predicts downstream drift and environment corruption 3 steps before they become observable!
<code>`findings/idf_table.json`</code> • 5,454-term pre-computed Inverse Document Frequency table built from 5,013 trajectory steps.	• Powers LocalConceptIndex. • Computes sparse TF-IDF vectors for task descriptions without needing external neural embedding models.	When a new task starts: • Compares task description against historical failed tasks. • Injects known pitfall advisories during on_plan_proposed().

TRACE WALKTHROUGH & EMPIRICAL PROOF

Live Concrete Trace & Empirical Verification

Tracing a dangerous action through the causal graph to preventive steering in 0.41 milliseconds.



Benchmark: Latency & Speed	Benchmark: Memory Footprint	Repository Test Suite Verification
• Target latency budget: < 15.0 ms. • Actual HippoRAG execution: 0.38 - 0.52 ms. • 30x faster than budget, introducing zero perceptible delay to agent execution.	• In-process NetworkX graph requires < 8 MB RAM. • Zero background database daemons. • Supports both in-memory testing (:memory:) and local JSON persistence.	• 25 / 25 dedicated memory system unit tests passing. • 184 / 184 (100%) full project tests passing. • Zero regressions introduced across the entire codebase.